DATUBĀZU MODELĒŠANA UN NORMALIZĀCIJA

Daniels Mēters
Sofia Lucenko

JĒDZIENI

Datu bāžu modelēšana ir process, kurā mēs plānojam, kā dati tiks glabāti datu bāzē, lai tie būtu:

- saprotami,
- ērti lietojami,
- bez liekas datu atkārtošanās.

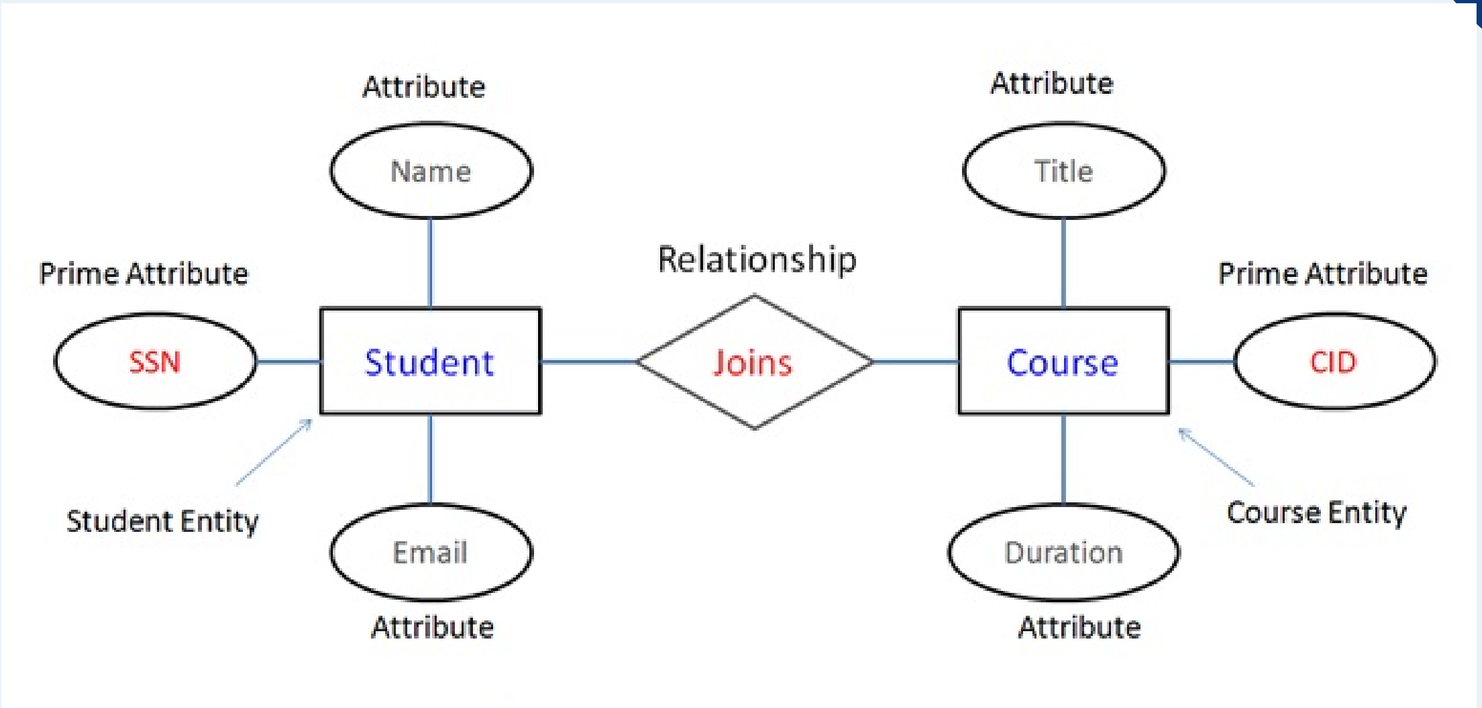
Normalizācija ir process, kurā mēs sakārtojam tabulas, lai:

- nebūtu liekas informācijas,
- nerastos kļūdas datu labošanā,
- dati būtu loģiski sadalīti.

JĒDZIENI

ER diagramma ir vizuāls zīmējums, kas parāda:

- **Entītijas** (objekts vai jēdziens),
- **Atribūtus** (entītijas īpašības),
- **Attiecības starp entītijām** (parāda, kā entītijas ir saistītas)



NORMĀLĀS FORMAS

Normālā forma ir noteikumu kopums, kas palīdz pareizi sakārtot datus tabulās.
Katrs nākamais solis novērš konkrētu problēmu.

Normālizcija soļi:

- NF1
- NF2
- NF3

NORMĀLĀS SOLIS: NF1

Ko tā risina?
Atkārtojošos laukus un sarakstus vienā kolonnā.

Noteikumi:

- Katrā lauciņā viena vērtība
- Nav masīvu, sarakstu vai vairāku vērtību vienā kolonnā
- Katram ierakstam ir unikāla atslēga (ID)

Pirms:

Skolēns	Telefoni
Anna	12345, 67890

Pēc:

Skolēna_ID	Skolēns	Telefons
1	Anna	12345
1	Anna	67890

NORMĀLĀS SOLIS: NF2

Ko tā risina?

Daļējas atkarības no primārās atslēgas.

Noteikumi:

- Tabula jau ir NF1
- Katrs atribūts ir saistīts tikai ar savo primāro atslēgu

Pirms:

Skolēna_ID	Kursa_ID	Skolēna_vārds	Kursa_nosaukums
1	101	Anna	Matemātika

Pēc:

Skolēna_ID	Vārds
1	Anna

Skolēna_ID	Kursa_ID
1	101

Kursa_ID	Nosaukums
101	Matemātika

NORMĀLĀS SOLIS: NF3

Ko tā risina?

Tranzitīvās atkarības

Noteikumi:

- Tabula jau ir NF2
- Ja $A \rightarrow B$ un $B \rightarrow C$, tad $A \nrightarrow C$

Pirms:

CAND_NO	CAND_NAME	CAND_STATE	CAND_COUNTRY	CAND_AGE
1	Amayra	West Bengal	India	18
2	Rihaan	Haryana	India	17
3	Manya	Haryana	India	19

Pēc:

CAND_NO	CAND_NAME	CAND_STATE	CAND_AGE
1	Amayra	West Bengal	18
2	Rihaan	Haryana	17
3	Manya	Haryana	19

CAND_STATE	CAND_COUNTRY
West Bengal	India
Haryana	India

TABULU SAISTĪBAS

Viens pret vienu (1:1)

- Vienam ierakstam atbilst viens ieraksts citā tabulā
- Piemērs: Persona - Pase

Viens pret daudziem (1:N)

- Vienam ierakstam var būt daudzi saistītie ieraksti
- Piemērs: Skolotājs - Kursi

Daudzi pret daudziem (M:N)

- Daudzi ieraksti saistīti ar daudziem
- Piemērs: Skolēni - Kursi
- Tiek veidota starp-tabula (piem., Pieraksts)

UZDEVUMS:

Izmmantojot normalizācijas soļus izlabot datu bāžu tabulas.

Atgādne:

Ko tā risina?(NF1)
Atkārtojošos laukus un sarakstus vienā kolonnā.

Pirms:

Skolēns	Telefoni
Anna	12345, 67890

Pēc:

Skolēna_I	Skolēns	Telefons
1	Anna	12345
1	Anna	67890

Ko tā risina?(NF2)
Daļējas atkarības no primārās atslēgas.

Pirms:

Skolēna_ID	Kursa_ID	Skolēna_vārds	Kursa_nosaukums
1	101	Anna	Matemātika

Pēc:

Skolēna_I	Vārds
1	Anna

Skolēna_I	Kursa_ID
1	101

Kursa_ID	Nosaukums
101	Matemātika

Ko tā risina?(NF3)
Daļējas atkarības no primārās atslēgas.

Pirms:

Skolēna_ID	Kursa_ID	Skolēna_vārds	Kursa_nosaukums
1	101	Anna	Matemātika

Pēc:

Skolēna_	Vārds
1	Anna

Skolēna_	Kursa_ID
1	101

Kursa_ID	Nosaukums
101	Matemātika

ATBILDES:

1.uzd

Skolēna_ID	Skolēns	Telefons
1	Anna	12345
1	Anna	67890
2	Jānis	55555

2.uzd

Pasūtījuma_ID	Prece
101	Maize
101	Piens
101	Sviests
102	Sula

ATBILDES:

3.uzd

Skolēna_ID	Vārds
1	Anna
2	Jānis

Kursa_ID	Nosaukums
101	Matemātika

Skolēna_ID	Kursa_ID
1	101
2	101

4.uzd

Darbinieka_ID	Vārds
1	Laura
2	Marks

Nodaļas_ID	Nosaukums
10	IT

Darbinieka_ID	Nodaļas_ID
1	10
2	10

ATBILDES:

5.uzd

Skolēna_ID	Vārds	Pilsēta
1	Anna	Rīga
2	Jānis	Rīga

Pilsēta	Valsts
Rīga	Latvija

6.uzd

Skolēna_ID	Vārds
1	Anna
2	Jānis
3	Laura

Kurss	Skolotājs
Matemātika	Ozols
Fizika	Kalniņa

Skolotājs	Kabinets
Ozols	203
Kalniņa	305

Skolēna_ID	Kurss
1	Matemātika
1	Fizika
2	Matemātika
3	Fizika

ATBILDES:

7.uzd

Pasūtījuma_ID	Klients
1001	Marta
1002	Jānis
1003	Marta

Pasūtījuma_ID	Prece
1001	Maize
1001	Piens
1002	Sula
1003	Maize
1003	Sula

Piegādātājs	Pilsēta
FreshFood	Rīga
DrinkCo	Jelgava

Prece	Piegādātājs
Maize	FreshFood
Piens	FreshFood
Sula	DrinkCo

UZDEVUMS:

Situācijas apraksts

Skolā tiek uzturēta sistēma, kurā:

- skolēni var pieteikties kursiem,
- katru kursu vada viens skolotājs,
- skolēns var apmeklēt vairākus kursus,
- vienu kursu var apmeklēt vairāki skolēni.

1. daļa – Tabulu izveide

Students

- id (int)
- name(varchar)
- email(varchar)

Courses

- id (int)
- **title(varchar)**
- **teacher_id(int)**

Enrollments

- student_id (int)
- course_id(int)
- enrolled_at(date)

Teachers

- id (int)
- name(varchar)

2. daļa – Relāciju izveide

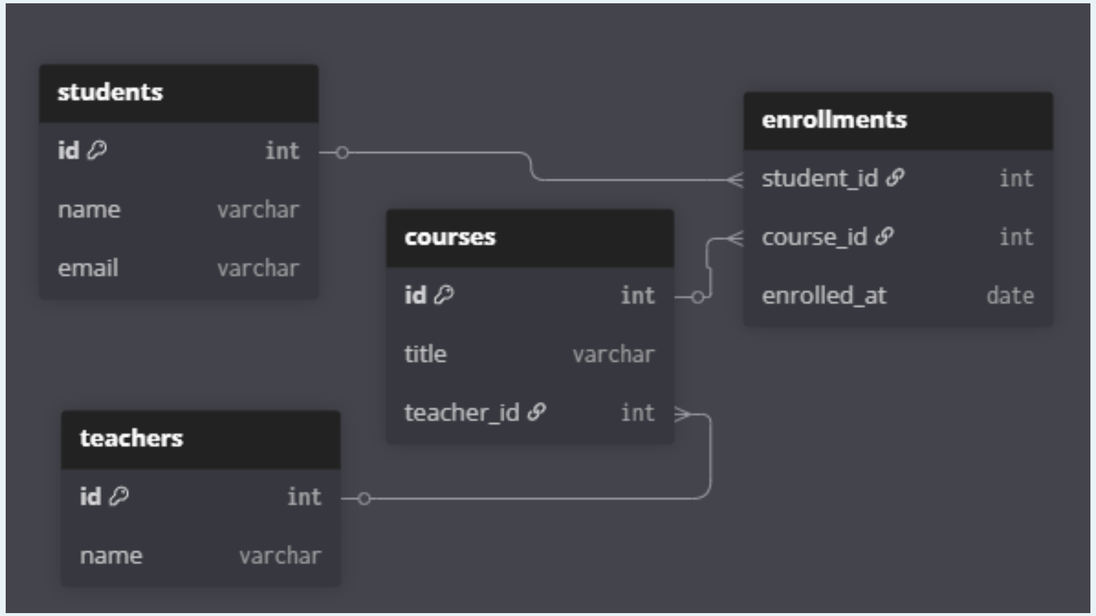
teachers → coursesid

- viens skolotājs vada daudzus kursus
- relācija: 1:N

students ↔ courses

- skolēns var apmeklēt vairākus kursus
- kursu var apmeklēt daudzi skolēni
- relācija: M:N, izmantojot tabulu enrollments

ATBILDES:




```
Table students {
  id int [pk]
  name varchar
  email varchar
}

Table teachers {
  id int [pk]
  name varchar
}

Table courses {
  id int [pk]
  title varchar
  teacher_id int
}

Table enrollments {
  student_id int
  course_id int
  enrolled_at date
}

Ref: teachers.id < courses.teacher_id
Ref: students.id < enrollments.student_id
Ref: courses.id < enrollments.course_id
```

UZDEVUMS:

Izveidot datu bāzi MySQL valodā, izmantojot phpMyAdmin, pēc izveidotas ER diagramas.

```
Table students {
  id int [pk]
  name varchar
  email varchar
}

Table teachers {
  id int [pk]
  name varchar
}

Table courses {
  id int [pk]
  title varchar
  teacher_id int
}

Table enrollments {
  student_id int
  course_id int
  enrolled_at date
}

Ref: teachers.id < courses.teacher_id
Ref: students.id < enrollments.student_id
Ref: courses.id < enrollments.course_id
```

Piemērs:

dbdiagram.io

```
Table users {
  id int [pk]
  name varchar
}

Table projects {
  id int [pk]
  title varchar
  owner_id int
}

Table user_projects {
  user_id int
  project_id int
}

Ref: users.id < projects.owner_id
Ref: users.id < user_projects.user_id
Ref: projects.id < user_projects.project_id
```

MySQL

```
CREATE TABLE users (
    id INT PRIMARY KEY,
    name VARCHAR(255)
);

CREATE TABLE projects (
    id INT PRIMARY KEY,
    title VARCHAR(255),
    owner_id INT,
    FOREIGN KEY (owner_id) REFERENCES users(id)
);

CREATE TABLE user_projects (
    user_id INT,
    project_id INT,
    PRIMARY KEY (user_id, project_id),
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (project_id) REFERENCES projects(id)
);
```

ATBILDE:

```sql
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(255),
    email VARCHAR(255)
);

CREATE TABLE teachers (
    id INT PRIMARY KEY,
    name VARCHAR(255)
);

CREATE TABLE courses (
    id INT PRIMARY KEY,
    title VARCHAR(255),
    teacher_id INT,
    FOREIGN KEY (teacher_id) REFERENCES teachers(id)
);

CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    enrolled_at DATE,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(id),
    FOREIGN KEY (course_id) REFERENCES courses(id)
);
```

PALDIES PAR UZMANBU!